

Installation Manual

GUI-Based Static and Modal Postprocessing Extension of a 2D Frame-Truss Direct Stiffness Solver

Imran Shahriar - 2735835

19 June 2026

Project: GUI-Based Static and Modal Postprocessing Extension of a 2D Frame-Truss Direct Stiffness Solver

Course: CE4011 Structural Analysis Software Development

Author: Imran Shahriar - 2735835

Python environment used: CE4011, Python 3.11

1. Purpose

This manual explains how to install, verify, and launch the CE4011 static/modal structural analysis program. The software is a Python-based educational 2D frame-truss Direct Stiffness Method package with a Tkinter/Matplotlib graphical interface, static analysis, modal analysis, postprocessing, generated frame models, result tables, and selected linear modal RHA/RSA utilities.

The program is intended for educational structural-analysis workflows. It is not a commercial structural design package.

2. System requirements

Recommended environment:

- Windows 10 or Windows 11.
- Anaconda or Miniconda.
- Python 3.11.
- A working Tkinter installation.
- Required Python packages, normally installed from `requirements.txt` if included.

Core package categories:

- `numpy` for numerical arrays.
- `scipy` for eigenvalue and sparse/numerical routines.
- `matplotlib` for plots and embedded GUI figures.
- `pytest` for verification tests.
- Python standard-library modules including `tkinter`, `json`, `xml`, `csv`, and `pathlib`.

3. Project folder

The project should be run from the repository root. Example development path:

```
C:\DDrive\Imran\METU\Coursework\CE4011\Project
```

Running scripts from another folder may cause relative paths for inputs, examples, generated models, or result directories to fail.

4. Conda environment setup

If the prepared CE4011 environment already exists, skip to Section 5. Otherwise, create and activate a new environment:

```
conda create -n CE4011 python=3.11 -y
conda activate CE4011
```

Install dependencies from the repository, if requirements.txt is available:

```
python -m pip install -r requirements.txt
```

If no requirements file is provided, install the essential packages manually:

```
python -m pip install numpy scipy matplotlib pytest
```

Tkinter usually comes with the standard Python distribution on Windows/Anaconda. If the GUI does not launch and the error mentions tkinter, install a Python distribution that includes Tk support.

5. Launching the GUI

From the project root:

```
cd "C:\\DDrive\\Imran\\METU\\Coursework\\CE4011\\Project"
C:\\Users\\Imran\\anaconda3\\envs\\CE4011\\python.exe scripts\\run_static_gui.py
```

If the environment is already activated, this shorter command is sufficient:

```
python scripts\\run_static_gui.py
```

The main window should open with the title similar to:

```
CE4011 Static/Modal Result Viewer
```

6. Basic verification commands

Run these commands from the project root before submission or handoff:

```
C:\\Users\\Imran\\anaconda3\\envs\\CE4011\\python.exe -m pytest -q
C:\\Users\\Imran\\anaconda3\\envs\\CE4011\\python.exe scripts\\run_gui_backend_verification.py
C:\\Users\\Imran\\anaconda3\\envs\\CE4011\\python.exe scripts\\run_final_core_smoke.py
```

Expected final results:

```
347 passed
GUI backend verification: Total: 13, PASS: 13, FAIL: 0
Final core smoke: all checks PASS
```

A Windows temporary-directory cleanup warning after pytest may appear. If the tests themselves pass, this warning is not treated as a solver failure.

7. Optional verification and example scripts

The following scripts generate example result files and figures under results/:

```
python scripts\\generate_proposal_models.py
python scripts\\run_member_load_examples.py
python scripts\\run_station_postprocessing_case.py
python scripts\\run_solver_diagnostics_case.py
python scripts\\compare_braced_unbraced.py
python scripts\\run_modal_rha_case.py
python scripts\\run_modal_rsa_case.py
```

The RHA/RSA scripts are supplementary linear modal postprocessing examples. They are not nonlinear seismic design workflows.

8. Generated outputs

Most generated outputs are written under:

results/

Common output folders include:

- `results/final\_core\_smoke/`
- `results/member\_load\_examples/`
- `results/station\_postprocessing/`
- `results/solver\_diagnostics/`
- `results/comparisons/`
- `results/rha/`

Generated outputs are normally ignored by version control unless selected figures or CSV files are intentionally included for the report.

9. Input files and companion mass files

The program supports JSON, XML, text-deck, form-based GUI input, and generated parametric frame models. Modal masses may be stored in the model file or in a companion mass file. For example:

```
model_a_5story_unbraced.json  
model_a_5story_unbraced_masses.json
```

When opening generated or saved modal models, ensure the companion mass file remains in the expected location if the model depends on explicit modal masses.

10. Troubleshooting

GUI does not launch

Check that you are running from the project root and using the CE4011 Python environment. Also confirm that Tkinter is available.

Import error or circular import message

Run from the project root, not from inside src/. Then run:

```
python -m pytest tests\test_import_stability.py -q
```

Model opens but modal analysis fails

Check that the model has modal mass data. For lateral frame examples, at least one free translational DOF must have assigned mass. If a companion mass file is missing, restore it or define masses through the GUI.

Static deformed shape looks undeformed

The numerical displacement may be small. Use the autoscaled deformed-shape plot or increase the static deformation display scale. Actual displacement values should be read from the result tables.

Force diagram sign differs from Ftool

This is usually a display convention issue. The program keeps numerical internal-force values separate from the plotted side/sign convention.

Units appear wrong

The unit system is metadata and labeling only. Changing the displayed unit convention does not rescale existing numerical input values.

11. Submission package recommendation

Recommended final package contents:

```
PROJECT_REPORT.pdf
INSTALLATION_MANUAL.pdf
USER_MANUAL.pdf
README.md
video_link.txt
source_code/ or GitHub_link.txt
inputs/
selected_results/
verification_outputs/
```

The report and manuals should state that the program is a linear elastic 2D frame-truss educational analysis package.